



INTERNAL

Deployment Guide

Quenyx vOPS HUB — v1.0.0 RC1 · Document v2.0

Document Version	2.0
Software Version	v1.0.0 RC1
Applies To	Quenyx vOPS HUB v1.0.0 RC1
Classification	Internal
Owner	DevOps / SRE
Status	Released
Last Updated	2026-06-29
Document Type	Deployment guide

REVISION HISTORY

Version	Date	Notes
1.0	2026	Initial v1 pack (through Sprint 19).
2.0	2026-06-29	Aligned to v1.0.0 RC1; native monitoring scheduler (`observe:run-checks` / `observe:evaluate-alerts`).

Table of Contents

1. Prerequisites	0
2. Ubuntu deployment (single node)	0
3. Laravel backend	0
4. React frontend	0
5. Node gateway	0
6. Nginx + SSL	0
7. Environment variables (key ones)	0
8. Queues	0
9. Scheduler	0
10. Cache	0
11. AI / RAG / Copilot environment (opt-in)	0
12. Storage	0
13. Database & migrations	0
14. Seeders & corpus	0
15. Backup & restore	0
16. Rollback	0
17. Health checks	0
18. Production checklist	0

10 — Deployment Guide

Audience: DevOps, implementation partners. **Canonical source:** This consolidates the repo-root `DEPLOYMENT.md` and adds QCIF/AI specifics. Where the two differ, the root `DEPLOYMENT.md` and the actual `.env.example` prevail. **Host-specific values below are examples — substitute your own.**

1. Prerequisites

Component	Version
PHP	8.3 recommended (8.1+ supported); extensions: <code>mbstring</code> , <code>pdo_mysql</code> , <code>openssl</code> , <code>curl</code> , <code>fileinfo</code> , <code>bcmath</code>
Composer	2.x
Node.js / npm	18+ LTS or 20+ / npm 9+ (<code>npm ci</code>)
MySQL	8.0+
Nginx	reverse proxy + static frontend

2. Ubuntu deployment (single node)

Single host runs: Laravel backend (PHP-FPM or `artisan serve`), Node gateway, Nginx (serves `frontend/dist`, proxies `/api/`), MySQL, queue worker, and the Laravel scheduler.

```
git clone <repo-url> && cd quenyx-saas
mysql -u root -p < scripts/mysql-quenyx-setup.sql # first-time DB setup
```

3. Laravel backend

```
cd backend
composer install --no-dev --optimize-autoloader
cp .env.example .env
# Edit DB_*, APP_URL, APP_KEY, SEED_ADMIN_PASSWORD, GATEWAY_INTERNAL_SECRET, AI/RAG vars (see §1)
php artisan key:generate
php artisan migrate --force
php artisan db:seed --force
php artisan config:cache && php artisan route:cache && php artisan view:cache
```

4. React frontend

```
cd ../frontend
npm ci
# set VITE_API_BASE_URL to your API base (same-origin /api recommended)
npm run build      # output: frontend/dist
```

5. Node gateway

```
cd ../gateway
npm ci
npm run build      # rebuild + restart after every change
# env: GATEWAY_PORT=4000, BACKEND_BASE_URL=http://127.0.0.1:8000, ENTITLEMENTS_CACHE_TTL_MS=3000
#      GATEWAY_INTERNAL_SECRET=<strong shared secret, also set in backend>
```

6. Nginx + SSL

- Document root → `frontend/dist` ; SPA fallback `try_files $uri /index.html` .
- `location /api/` → `proxy_pass http://127.0.0.1:4000` (gateway), forwarding `Authorization` .
- **SSL**: terminate TLS at Nginx (or LB); redirect HTTP→HTTPS. (See root `DEPLOYMENT.md` for full server blocks, single-node and multi-node.)

7. Environment variables (key ones)

Var	Purpose
<code>APP_ENV=production</code> , <code>APP_DEBUG=false</code> , <code>APP_KEY</code> , <code>APP_URL</code>	Core (debug must be false in prod)
<code>DB_DATABASE</code> / <code>USERNAME</code> / <code>PASSWORD</code>	MySQL
<code>QUEUE_CONNECTION=database</code>	Queue worker
<code>SANCTUM_STATEFUL_DOMAINS</code>	CORS/auth domains
<code>SEED_ADMIN_PASSWORD</code>	Required before seeding <code>UserSeeder</code>
<code>GATEWAY_INTERNAL_SECRET</code>	Shared backend↔gateway secret
<code>GATEWAY_BASE_URL</code>	Public gateway URL for agent enrollment

8. Queues

Port scans and **RAG indexing jobs** require a worker:

```
# /etc/systemd/system/quenyx-queue.service (ExecStart)
/usr/bin/php artisan queue:work --sleep=3 --tries=3 --max-time=3600
```

9. Scheduler

QynSight checks require cron running the Laravel scheduler:

```
* * * * * cd /path/backend && php artisan schedule:run >> storage/logs/scheduler.log 2>&1
```

Without it, monitoring shows "Last poll: never".

10. Cache

`config:cache`, `route:cache`, `view:cache` in production. Use Redis for cache/sessions in multi-node (optional). Run `config:clear` after `.env` changes in dev.

11. AI / RAG / Copilot environment (opt-in)

All AI is **off by default**. Enable deliberately:

Var	Default	Effect
<code>AI_PROVIDER</code>	<code>mock</code>	<code>openai</code> to use real models
<code>AI_ENABLED</code>	<code>false</code>	master switch for real model calls
<code>OPENAI_API_KEY</code> / <code>OPENAI_MODEL</code> / <code>OPENAI_EMBEDDINGS_MODEL</code>	unset	provider config (models from env only)
<code>COPILOT_ENABLED</code>	<code>true</code>	Copilot on (still mock unless <code>AI_ENABLED</code>)
<code>AI_PROMPT_LOGGING_ENABLED</code>	<code>false</code>	store prompt content (keep off)
<code>AI_CONVERSATION_PERSISTENCE_ENABLED</code>	<code>false</code>	persist conversations (keep off)
<code>AI_COPILOT_RAG_ENABLED</code> / <code>RAG_ENABLED</code> / <code>EMBEDDINGS_ENABLED</code>	<code>false</code>	RAG runtime
<code>VECTOR_PROVIDER</code>	unset	<code>openai</code> (metadata-only today)
<code>RAG_INDEX_TENANT_EVIDENCE</code>	<code>false</code>	keep off (privacy)

12. Storage

Ensure `backend/storage/app/agents` exists and is writable by the web user (agent binaries + on-demand Go build). Standard Laravel `storage/` and `bootstrap/cache` permissions apply.

13. Database & migrations

```
php artisan migrate --force
php artisan migrate:status # verify all applied
```

14. Seeders & corpus

```
php artisan db:seed --force # core seeders (set SEED_ADMIN_PASSWORD)
php artisan db:seed --class=ComplianceCorpusSeeder --force
php artisan compliance:seed-source-documents \
  database/corpus/nca/ecc-2-2024/source-documents.json \
  --framework=nca-ecc --release=2:2024
```

Expected corpus: **5 domains, 108 controls, 108 requirements**, active **Revision v1**.

15. Backup & restore

- **DB:** `mysqldump` nightly; store offsite/encrypted. Restore: `mysql < dump.sql`.
- **Storage:** back up `backend/storage/` (uploaded artifacts, agent binaries).
- **Config:** keep `.env` in a secrets manager (never in git).

16. Rollback

- **App:** `git checkout <previous-tag>` → rebuild backend/frontend/gateway → restart services.
- **DB:** restore from the pre-deploy dump. **Do not** rely on `migrate:rollback` for destructive changes in production; restore from backup.
- **Corpus:** roll back to the prior active revision (deterministic snapshots; see Doc 18).

17. Health checks

- Backend: `GET /api/health`.
- Gateway: `GET /health` → `{"status":"ok","service":"gateway"}` (and `/ready`).
- Monitor ports 8000 (backend) and 4000 (gateway); alert on 502/503 and DB connectivity.

18. Production checklist

- ☐ `APP_ENV=production`, `APP_DEBUG=false`, strong `APP_KEY`, HTTPS `APP_URL`.
- ☐ `config:cache` + `route:cache` after deploy.
- ☐ CORS restricted to frontend origin; `SANCTUM_STATEFUL_DOMAINS` set.
- ☐ `GATEWAY_INTERNAL_SECRET` set on both sides.
- ☐ `SEED_ADMIN_PASSWORD` set; seeded credentials rotated.

- ☐ Queue worker + scheduler running.
- ☐ AI/RAG flags reviewed (off unless intended; tenant-evidence indexing off).
- ☐ Migrations applied; corpus counts verified.
- ☐ Backups scheduled.